

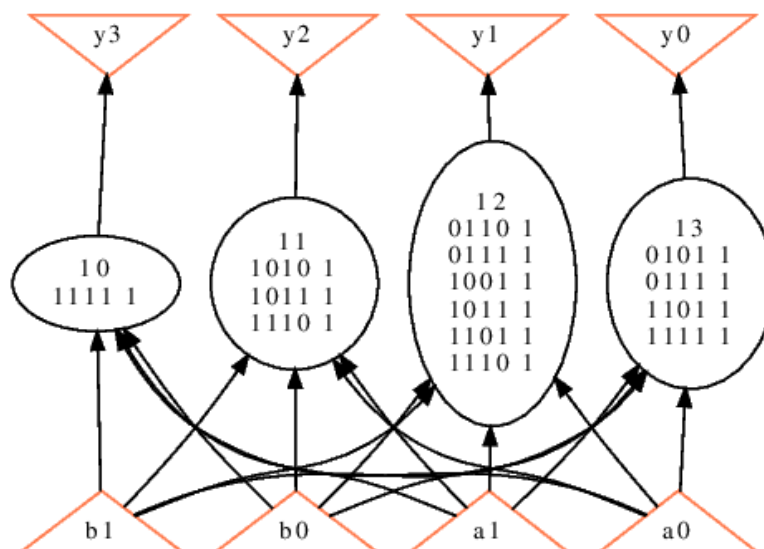


國立臺灣大學
National Taiwan University

Logic Synthesis and Verification

Programming Assignment 1

Matéo Lambert (藍馬修) - A15921201



Exercise 2

(a)

To represent a two-bit unsigned multiplier we need four inputs: two inputs for each bit of the two operands. Their names are $\{a1, a0\}$ for the first operand and $\{b1, b0\}$ for the second.

The outputs must be able to represent the biggest product possible: $11_2 \times 11_2 = 1001_2 = 9_{10}$. The biggest output is 9 so the output must be 4-bit long: $\{y3, y2, y1, y0\}$.

After computing the truth tables for each output, we can write the BLIF file.

```
.model mul
.inputs a1 a0 b1 b0
.outputs y3 y2 y1 y0
.names a1 a0 b1 b0 y3
1111 1
.names a1 a0 b1 b0 y2
1010 1
1011 1
1110 1
.names a1 a0 b1 b0 y1
0110 1
0111 1
1001 1
1011 1
1101 1
1110 1
.names a1 a0 b1 b0 y0
0101 1
0111 1
1101 1
1111 1
.end
```

Code 1: two-bit unsigned multiplier in BLIF format

(b)

Once the file is generated, ABC can be executed to read, analyse, and manipulate the logic module.

```

=====
abc 01> read mul.blif
abc 02> print_stats
mul                               : i/o =   4/   4 lat =   0 nd =   4 edge =   16 cube =   14 lev = 1
abc 02> show

```

Figure 3: ABC console

The `read <file>` command loads the logic network from the specified file into the current session.

`print_stats` is used to print various statistics depending on the model representation. The model is currently represented as a logic network so we have different information regarding the graph like the number of nodes, edges or cubes in the graph.

Finally `show` displays the graph:

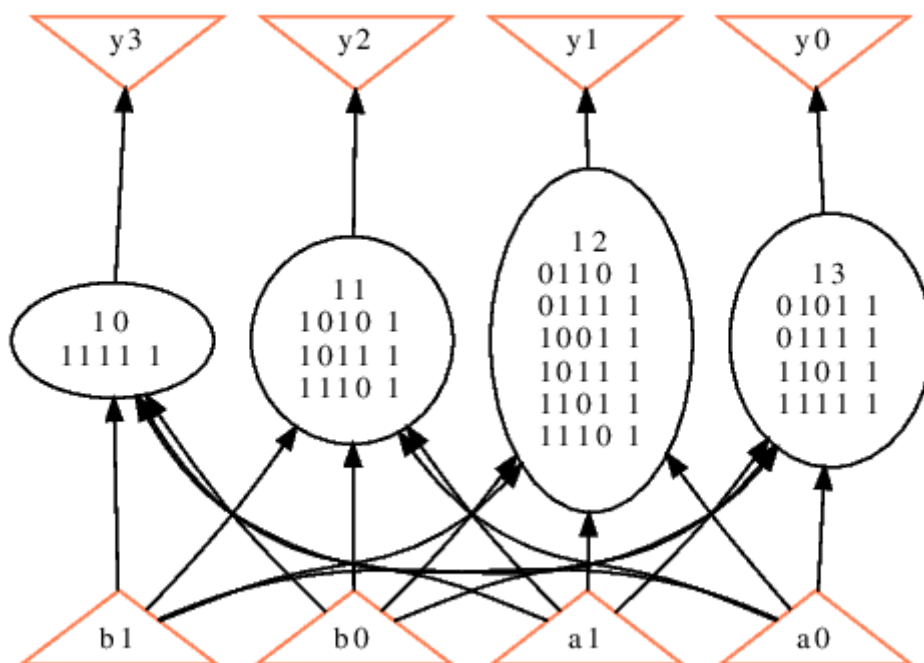


Figure 4: mul.blif in ABC as a logic network

ABC allows to change between representations really easily. For example the `strash` command converts transforms combinational logic into an AIG. We can visualise the change by using the `show` command again:

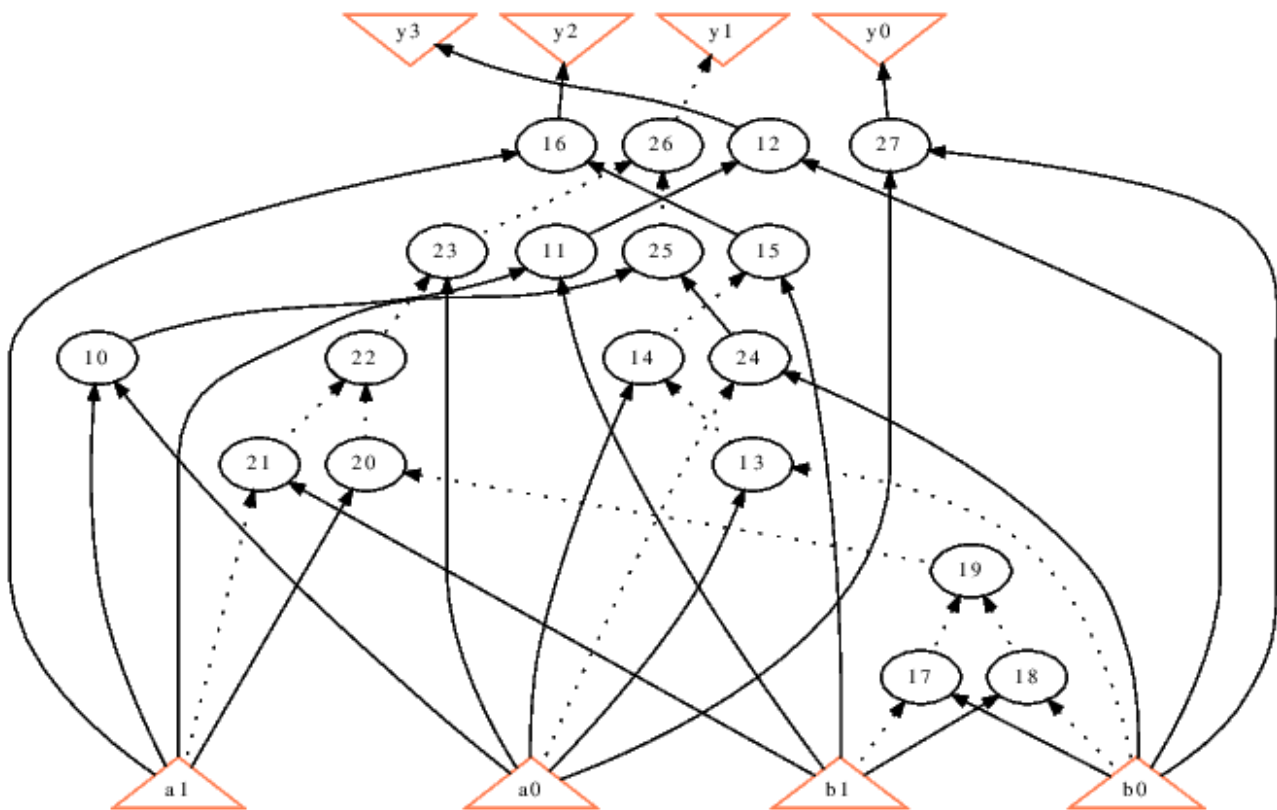


Figure 5: mul.blif in ABC as a AIG

And finally we can also use the `collapse` command to convert the graph into a BDD. This time to display the graph correctly we must use the `show_bdd -g` command.

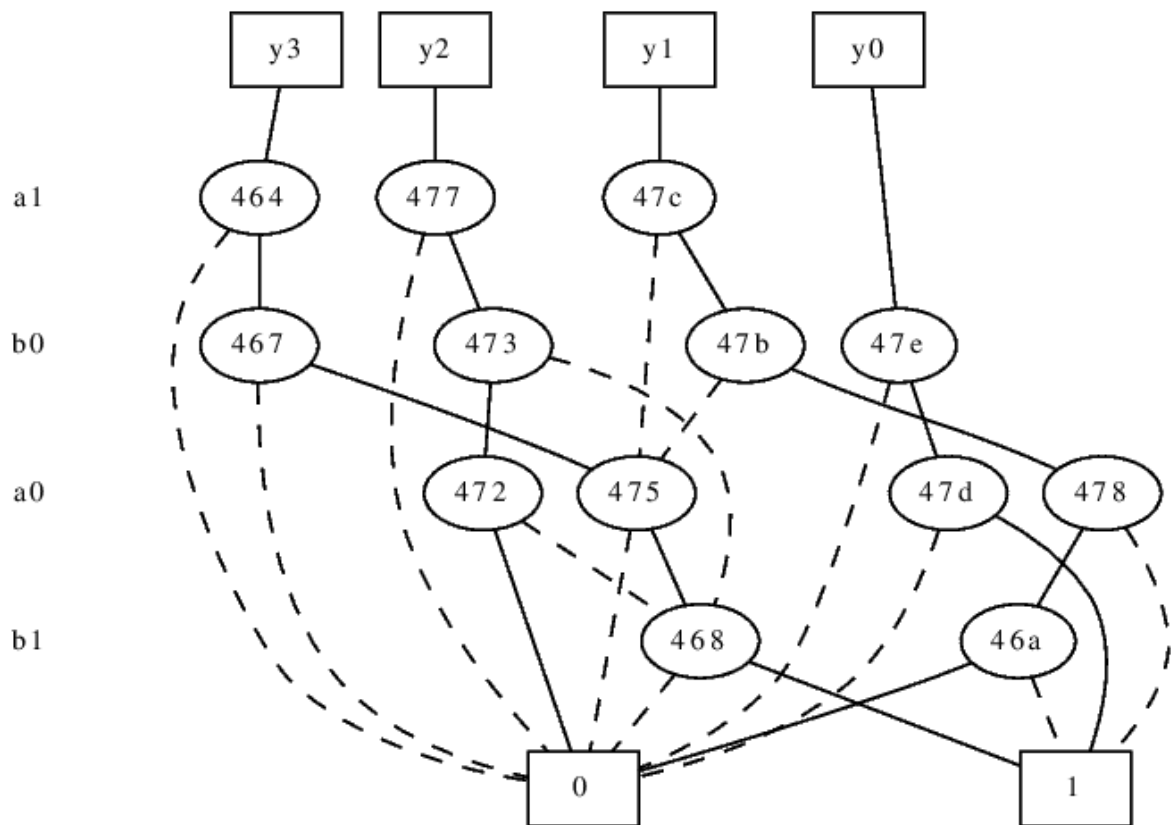


Figure 6: mul.blif in ABC as a BDD

Exercise 3

(a)

aig / strash

If we use the **show** command just after trying to convert the mul.blif logic network with the **aig** command ABC will display this graph:

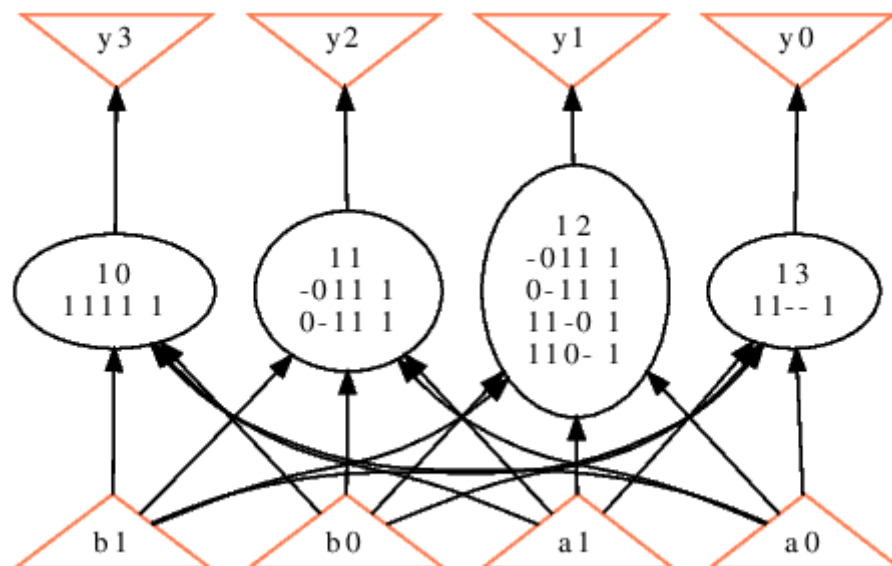


Figure 7: mul.blif after using the `aig` command

It seems that we still have the same representation. This is because the `aig` command does not convert the graph into a full AIG representation, it actually converts only the local functions of the nodes to AIGs.

On the other end the `strash` command converts the whole graph to an AIG. As can be seen in the Figure 5, the graph is not represented by a logic network with different nodes anymore, it is now a real AIG.

bdd / collapse

The difference between `bdd` and `collapse` is the same as between `aig` and `strash`. The `bdd` command converts only the local functions of the nodes in the logic network representation to BDDs while the `collapse` command converts the whole graph representation to a BDD.

(b)

Given a structurally hashed AIG we can combine the AIG gates into logic nodes to create a logic network with the `renode` command.

Once the graph is reshaped as a logic network we can use the `sop` command to convert the local functions of the nodes to SOPs.

```

=====
abc 01> read mul.blif
abc 02> strash
abc 03> print_stats
mul                               : i/o = 4/ 4 lat = 0 and = 18 lev = 6
abc 03> renode
abc 04> print_stats
mul                               : i/o = 4/ 4 lat = 0 nd = 8 edge = 18 aig = 14 lev = 3
abc 04> sop
abc 04> print_stats
mul                               : i/o = 4/ 4 lat = 0 nd = 8 edge = 18 cube = 11 lev = 3
abc 04>

```

Figure 8: Sequence of ABC commands to convert an AIG to a logic network with node function expressed in sum-of-products

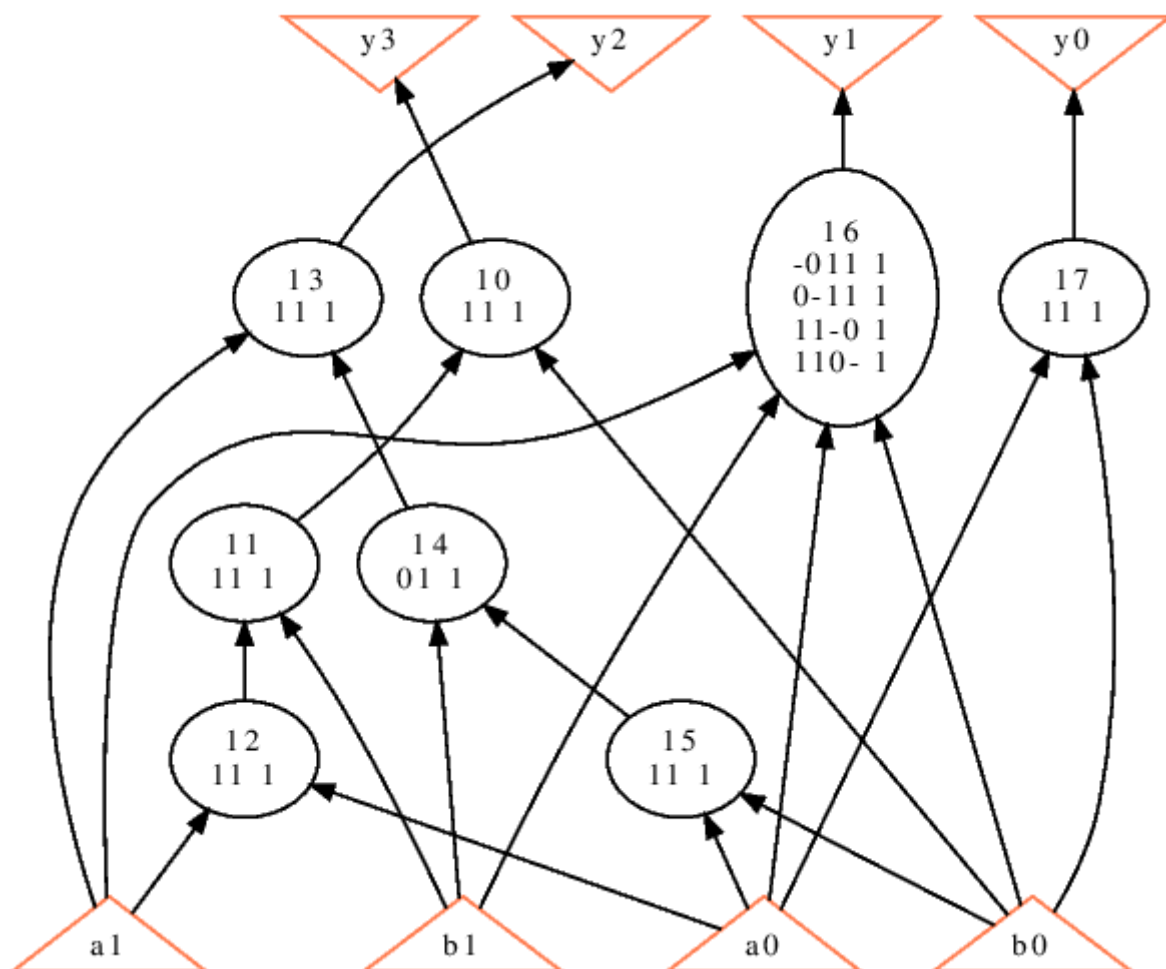


Figure 9: The resulting graph